

Module 5 Guided Lab Session 2: Design the Schema

Field	Value
Module	M5: Entity Framework Core 10 and PostgreSQL
Exercises	3 (GroupBy + pagination), 4 (EntityTypeConfiguration), 5 (Relationships)

Prerequisites Check Before You Start

Your TMS solution must contain:

- A working `TmsDbContext` registered with `UseNpgsql` and a valid `TmsDatabase` connection string
- An applied `InitialCreate` migration `dotnet ef migrations list` shows it as applied
- SQL logging enabled in development every query in Session 1 produced SQL in the console
- Working entities for `Student`, `Course`, and `Enrollment` with appropriate primary keys and value-typed properties

If any of these are missing, go back to Session 1. The exercises in this session evolve the schema you scaffolded yesterday without a working `DbContext` and visible SQL, the design choices you make today are invisible.

What This Session Is Really About

In Session 1 you proved the database can talk back. Today you design what it says.

The three exercises share one theme: **let the database do the work, and keep your C# code short and inspectable.**

1. **Paginate and group in SQL** (Exercise 3) never load a whole table just to take the first 20 rows
2. **Move every entity's mapping into its own class** (Exercise 4) `OnModelCreating` is a router, not a 200-line config dump
3. **Model the relationship graph deliberately** (Exercise 5) `OnDelete` behaviour, navigation properties, and FK direction are design choices, not defaults

These are the decisions that make the difference between an EF Core codebase that scales with the team and one that becomes the file nobody wants to open. Every senior interview asks at least one of: “How do you paginate?”, “Where do your entity configurations live?”, and “What happens when I delete a course that has enrollments?” Today you build defensible answers to all three.

Exercise 3: GroupBy, aggregates, and pagination

Context: The dashboard must show paged roster data and summary tiles. The database should do the math.

Tasks

1. Implement a paged list of students: **page size 20, page number** as a parameter, stable sort by **name** (always OrderBy before Skip/Take).
2. Implement a summary query: top **5** courses by enrollment count with course title and count (same idea as quiz-style reporting).

*// **TODO 1:** `Pagination OrderBy, Skip((page - 1) * pageSize), Take(pageSize), ToListAsync with CancellationToken.`*

*// **TODO 2:** `Top 5 courses by enrollment GroupBy, order by count, Take(5).`*

Checkpoint: SQL log shows LIMIT / OFFSET and GROUP BY where appropriate.

Troubleshooting:

- **Client evaluation warning (“could not be translated”):** Your GroupBy projection uses a method EF cannot translate to SQL. Simplify the lambda use only property access and aggregates (Count(), Average()) inside Select.
- **Pagination returns wrong rows:** You must OrderBy before Skip/Take. Without a stable sort, PostgreSQL may return rows in any order between pages.
- **Empty results:** Confirm you seeded enough enrollment rows in Exercise 2. Run SELECT COUNT(*) FROM "Enrollments" in pgAdmin to verify.

Exercise 4: IEntityTypeConfiguration for each entity

Context: OnModelCreating should stay short. Each entity gets its own configuration class.

Steps

1. Create StudentConfiguration, CourseConfiguration, and EnrollmentConfiguration implementing IEntityTypeConfiguration<T>.
2. Move key lengths, required fields, and relationship setup from attributes (if any) into Fluent API as your team prefers.
3. Ensure TmsDbContext uses:

modelBuilder.ApplyConfigurationsFromAssembly(typeof(TmsDbContext).Assembly);

4. Add a migration that reflects your refinements (for example dotnet ef migrations add RefineTmsModel) and **inspect** before database update.

```
// TODO: In EnrollmentConfiguration, configure FKs and required properties.  
// Example shape (adjust names to your model):  
// builder.HasOne(e => e.Student).WithMany(s => s.Enrollments).HasForeignKey(e => e.StudentId);  
// builder.HasOne(e => e.Course).WithMany(c => c.Enrollments).HasForeignKey(e => e.CourseId);
```

Checkpoint: Three configuration classes, no giant OnModelCreating method.

Troubleshooting:

- **“The entity type requires a primary key”:** Your configuration is missing builder.HasKey(e => e.Id) or EF cannot infer the key by convention. Name the property Id or EnrollmentId, or configure it explicitly.
- **Migration shows unexpected table drops:** You renamed a DbSet property or an entity class. EF interprets this as “drop old table, create new one.” Rename in a separate migration with migrationBuilder.RenameTable() instead.
- **ApplyConfigurationsFromAssembly finds nothing:** The configuration classes must be public and in the same assembly as TmsDbContext. Check the assembly reference.

Exercise 5: Model the TMS graph

Context: A student enrolls in many courses; a course has many students; **enrollment** stores grade and dates.

Tasks

1. Ensure **one-to-many** from Course to Enrollment and from Student to Enrollment.
2. Decide **delete behavior** (for example Restrict on course delete when enrollments exist) and document why in a one-sentence comment.
3. Add or adjust navigation properties so Include can load related data in Session 3.

```
// TODO: Choose OnDelete behavior and express it in Fluent API.  
// builder.HasMany(...).WithOne(...).OnDelete(DeleteBehavior.Restrict);
```

Checkpoint: Model builds; migration would create correct FK constraints (you will generate it in Session 3, Exercise 6).

Troubleshooting:

- **Cascade cycle detected:** PostgreSQL rejects multiple cascade paths to the same table. Use DeleteBehavior.Restrict on at least one side and handle deletion in your application code.
- **Navigation property is always null:** You declared the property but forgot Include() when querying. Without eager loading, navigation properties remain null by default (EF Core does not use lazy loading unless explicitly configured).

Why this session ends here

You have made three decisions that scale from a 10-row demo to a 10-million-row production database:

- **Pagination and aggregation run in SQL.** A request for page 50 does not load pages 1 to 49 first. A request for “top five courses” does not enumerate every enrollment in C# memory. Both stay constant-cost as your data grows.
- **Configuration is per-entity and code-discoverable.** When the registrar asks “what is the maximum length of a course title?”, the answer is one file: CourseConfiguration.cs. A new team member can read your model in five minutes instead of fifty.
- **Relationships are deliberate, not accidental.** OnDelete is set explicitly on the side that needs to fail loudly. Navigation properties exist where Include will need them. Your migrations from now on add foreign keys with the constraints you expect, not whatever EF guesses.

In Session 3 you generate the migration that proves these design choices work, fix the most common production performance bug (N+1), add audit and concurrency safety, and write the one-page Repository/UoW design summary that closes the module.

Session 2 Checkpoint

Before moving to Session 3, confirm all four:

- ☐ dotnet ef migrations add RefineTmsModel (or a similarly named migration) generates a file you have read; the SQL log for database update shows the column changes you expected from your IEntityTypeConfiguration classes
- ☐ Three configuration classes exist (StudentConfiguration, CourseConfiguration, EnrollmentConfiguration); OnModelCreating contains only modelBuilder.ApplyConfigurationsFromAssembly(...) and is otherwise empty or near-empty
- ☐ Your paginated student endpoint logs SQL with LIMIT 20 OFFSET ...; the top-5 courses endpoint logs SQL with GROUP BY and ORDER BY ... DESC LIMIT 5
- ☐ You can point at the line in your Fluent API where OnDelete(DeleteBehavior.Restrict) (or your chosen behaviour) is set, and explain in one sentence why a course with enrollments should not cascade-delete its students' records

If any of the four fail, stay in Session 2. Session 3's migration discipline, performance work, and design discussion all assume these design foundations hold.